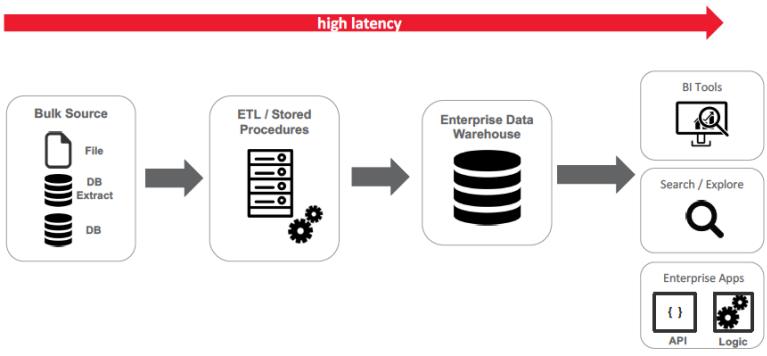
Chương 8
Kiến trúc dữ liệu lớn

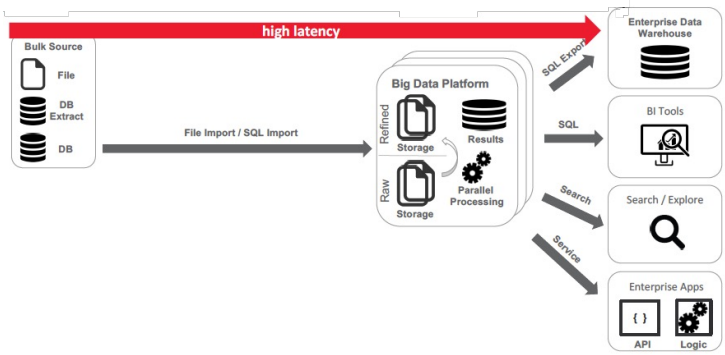
1

Traditional BI infrastructures



2

Hadoop solves Volume and Variety – not Velocity



3

Lambda Architecture

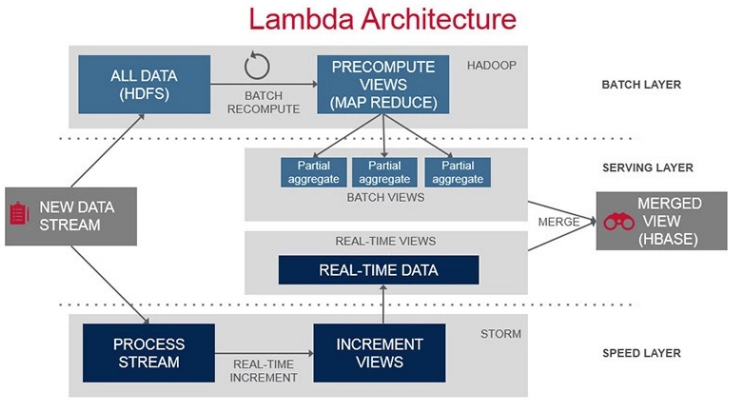
- A data-processing architecture designed to handle massive quantities of data by taking advantage of both batch and stream processing methods.
- Spark is one of the few data processing frameworks that allows you to seamlessly integrate batch and stream processing
  - Of petabytes of data
  - In the same application

I need fast access to historical data on the fly for predictive modeling with real time data from the stream



4

Lambda architecture



5

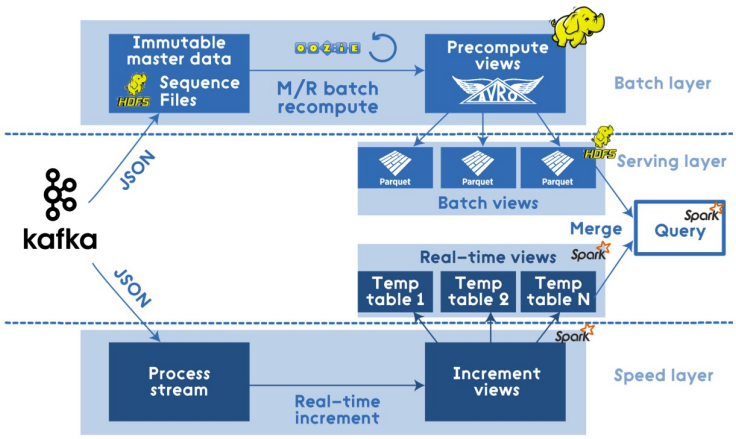
Relevance of data

query = function(batch view, real time view)
real time view = function(real time view, new data)
batch view = function(all data)



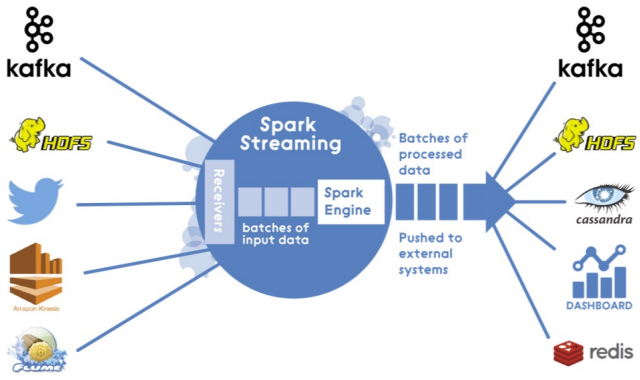
6

Lambda architecture: one implementation



7

Spark streaming



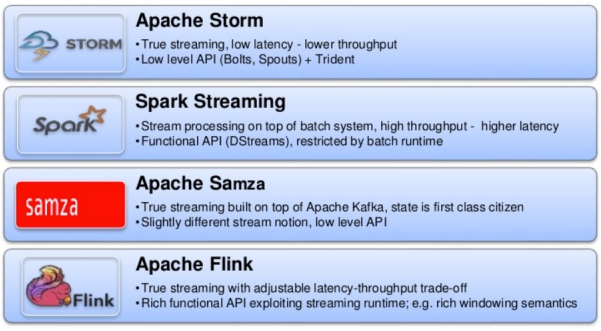
8

Spark streaming

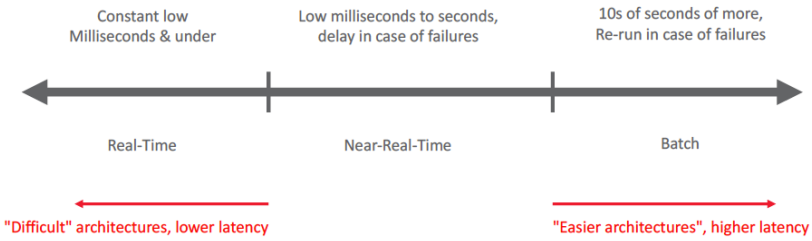
- Scalable, fault-tolerance stream processing system
- a streaming computation as: a series of very small, deterministic batch jobs
  - Chop up the live stream into batches of X seconds
  - Spark treats each batch of data as RDDs and processes them using RDD operations
  - Finally, the processed results of the RDD operations are returned in batches



Streaming landscape



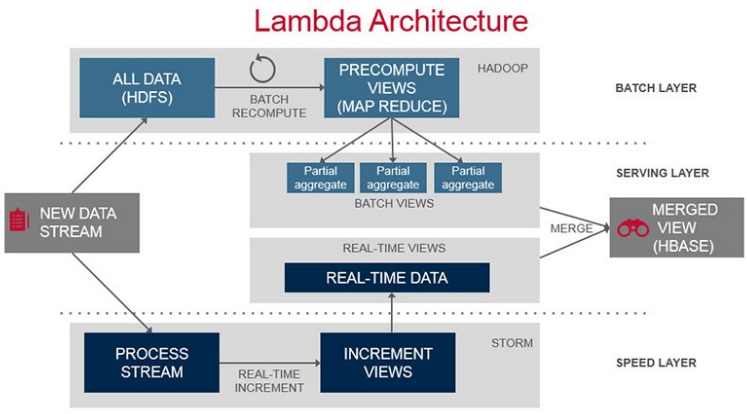
Stream vs. Batch processing



11

Case study

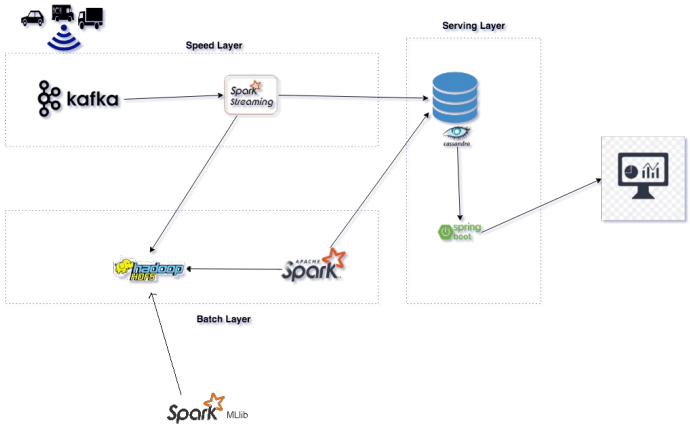
Tweets analysis using lambda architecture



12

Case study

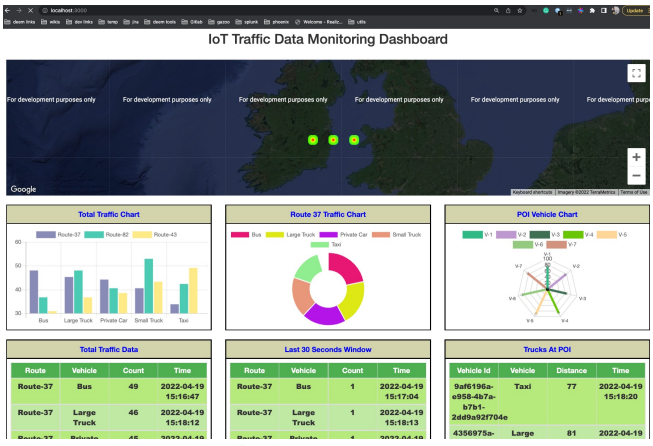
- A full big data pipeline (Lambda Architecture) with Spark, Kafka, HDFS and Cassandra.



13

Case study

- A full big data pipeline (Lambda Architecture) with Spark, Kafka, HDFS and Cassandra.



14

Batch Layer of project : To process data in batch we have used Apache Spark (fast and general engine for large-scale data processing) engine and to store batch view we have used cassandra. To do this we have created BatchProcessingUnit to create all batch view on master dataset.

```
class BatchProcessingUnit {


  val sparkConf = new SparkConf()
    .setAppName("Lambda_Batch_Processor").setMaster("local[2]")
    .set("spark.cassandra.connection.host", "127.0.0.1")
    .set("spark.cassandra.auth.username", "cassandra")


  val sc = new SparkContext(sparkConf)


  def start: Unit ={
    val rdd = sc.cassandraTable("master_dataset", "tweets")
    val result =
rdd.select("userid","createdat","friendscount").where("friendsCount
> ?", 500)

result.saveToCassandra("batch_view","friendcountview",SomeColumns("
userid","createdat","friendscount"))
    result.foreach(println)
  }
}
```

15

Speed Layer of project: In speed layer we have used Spark Streaming to process realtime tweets and store its view in cassandra.

To do this we have created SparkStreamingKafkaConsumer which read data from kafka queue “tweets” topic and send to view handler of speed layer to generate all view.

```
object SparkStreamingKafkaConsumer extends App {
  val brokers = "localhost:9092"
  val sparkConf = new
SparkConf().setAppName("KafkaDirectStreaming").setMaster("local[2]"
)
    .set("spark.cassandra.connection.host", "127.0.0.1")
    .set("spark.cassandra.auth.username", "cassandra")
  val ssc = new StreamingContext(sparkConf, Seconds(10))
  ssc.checkpoint("checkpointDir")
  val topicsSet = Set("tweets")
  val kafkaParams = Map[String, String]("metadata.broker.list" ->
brokers, "group.id" -> "spark_streaming")
  val messages: InputDStream[(String, String)] =
KafkaUtils.createDirectStream[String, String, StringDecoder,
StringDecoder](ssc, kafkaParams, topicsSet)
  val tweets: DStream[String] = messages.map { case (key, message)
=> message }
  ViewHandler.createAllView(ssc.sparkContext, tweets)
  ssc.start()
  ssc.awaitTermination()
```

16

Serving Layer of Project: In serving layer basically we have combined batch view data and realtime view data to satisfy adhoc query requirement.Here is an example in which have try to analyse all twitter users who match the specify hashtag and they have follower counts greater than 500 .

```
def findTwitterUsers(minute: Long, second: Long, tableName: String
= "tweets"): Response = {

  val batchInterval = System.currentTimeMillis() - minute * 60 *
1000

  val realTimeInterval = System.currentTimeMillis() - second * 1000

  val batchViewResult = cassandraConn.execute(s"select * from
batch_view.friendcountview where createdat >= ${batchInterval}
allow filtering;").all().toList

  val realTimeViewResult = cassandraConn.execute(s"select * from
realtime_view.friendcountview where createdat >=
${realTimeInterval} allow filtering;").all().toList

  val twitterUsers: ListBuffer[TwitterUser] = ListBuffer()

  batchViewResult.map { row =>

    twitterUsers += TwitterUser(row.getLong("userid"), new
Date(row.getLong("createdat")), row.getLong("friendscount"))

  }

  realTimeViewResult.map { row =>

    twitterUsers += TwitterUser(row.getLong("userid"), new
Date(row.getLong("createdat")), row.getLong("friendscount"))

  }

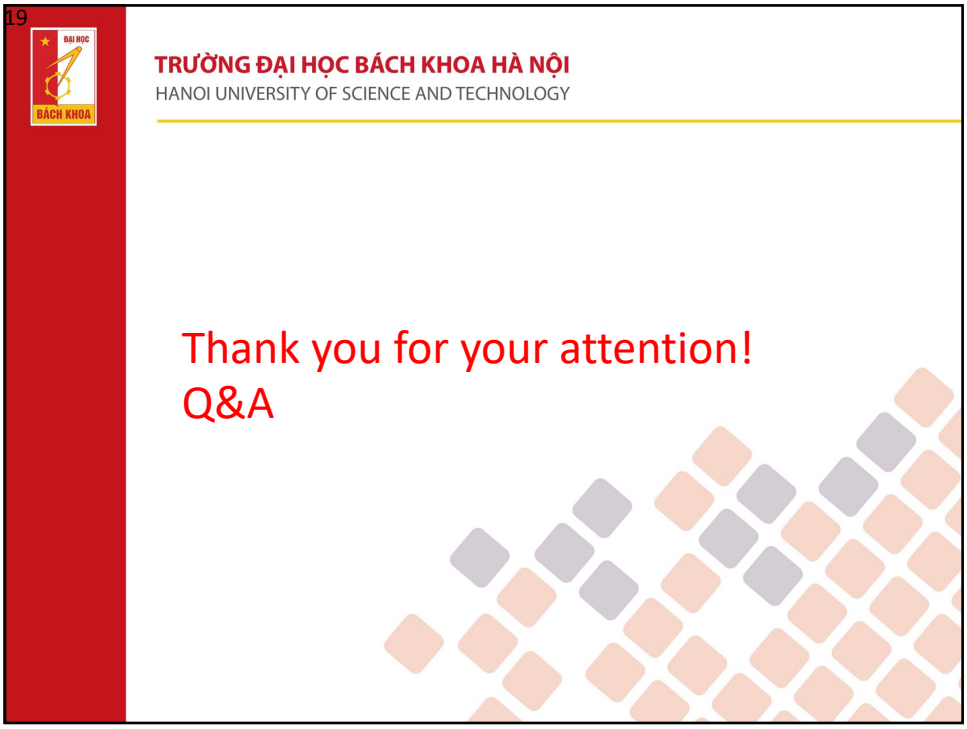
  Response(twitterUsers.length, twitterUsers.toList)

}
```

17

References

- https://github.com/OryxProject/oryx
- https://github.com/MicrosoftDocs/azure-docs/blob/master/articles/cosmos-db/lambda-architecture.md
- https://github.com/apssouza22/lambda-arch
- https://github.com/knoldus/Lambda-Arch-Spark


19
TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
Thank you for your attention!
Q&A
19